

Bases de données documentaires et distribuées, <http://b3d.bdpedia.fr>

Le modèle de données Cassandra

Cassandra organise les données en **tables** dénormalisées

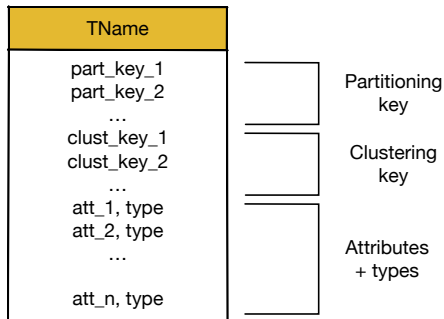
Principales différences avec le modèle relationnel

- Pas de clé étrangère (et pas de jointure)
- Imbrication (*nesting*) de valeurs structurées
- Pas d'indépendance logique/physique : la clé détermine le placement

Principe de conception

Pas de jointure, donc pas de symétrie d'accès : on doit organiser les données en fonction des accès applicatifs.

Structure d'un schéma



```
create table Tname (  
    part_key_1 type,  
    part_key_2 type,  
    ... type,  
    clust_key_1 type,  
    ... type,  
    att_1 type,  
    att_2 type,  
    ... type,  
    primary key (  
        (part_key_1, part_key2, ...),  
        (clust_key_1, clust_key_2, ...)  
    )  
)
```

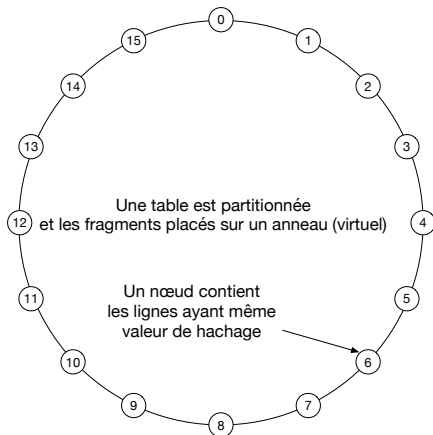
Imbrication de structures

C'est le principe de dénormalisation : on regroupe les données le plus possible dans des lignes pour éviter les jointures.

Un attribut attribut peut correspondre :

- à un ensemble de valeurs (non ordonnées) : **SET**
- à une liste de valeurs **LIST**
- à un dictionnaire : **MAP**
- à un tuple : **TUPLE**
- à une instance d'un type utilisateur

Clé de partitionnement = placement



- Pour chaque ligne on applique une fonction de hachage aux valeurs de la clé de partitionnement
- La valeur de hachage détermine le placement dans le système distribué
- $\Rightarrow$ une requête avec comme critère une clé de partitionnement ne concerne qu'un seul serveur

Clé de regroupement = contiguité

TName		
Part key	Clust key	(key, val) pairs
www.swh.org	aaa.html	...
www.swh.org	bird.jpg	...
...	...	...
www.swh.org	main.css	...
www.swh.org	...	...
www.swh.org	zeta.zip	...

- Dans un fragment (sur un serveur) les lignes sont triées sur la clé primaire
- Pour une même valeur de partitionnement, les lignes sont **consécutives** et ordonnées sur la clé de regroupement
- $\Rightarrow$ une requête sur une préfix de la clé primaire (part. + regroupement) correspond à un parcours **séquentiel** sur un seul nœud

Principes de conception

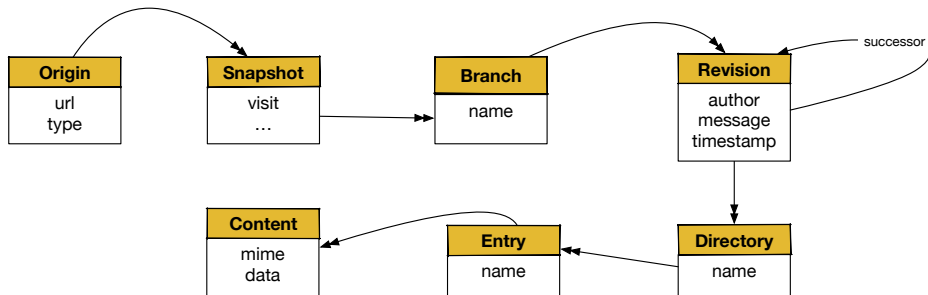
Très différent du relationnel. On adopte un des nombreux choix possible en fonction des besoins supposés.

- On conçoit les **chemins d'accès**, ou séquence des requêtes émises par une application
- On organise les tables pour que chaque requête s'exécute **localement** et **séquentiellement**
- On peut créer ponctuellement des index ou des vues matérialisées
- Au pire (mais inévitable ?) on multiplie les organisations physiques et donc la redondance

Pour quelle type d'application ?

Acceptable pour une approche WORM (*Write Once, Read many*)

Notre modèle



Commençons : cherchons les visites

Une première approche purement relationnelle ne fonctionne pas.

Schema

```
create table Origin
(
  origin_id uuid,
  url varchar,
  description text,
  primary key (origin_id))

create table Visit
(
  visit_id uuid,
  origin_id
  visit_ref int,
  date date,
  primary key (visit_id) )
```

Queries

Il faut deux requêtes (pas de jointure)

```
select origin_id in oid
from Origin
where url = 'sw.h.org'
```

```
select * from Visit
where origin_id = oid
```

Et Cassandra refuse les deux !

Mieux

Les critères de recherche doivent être dans la clé

```
create table Origin  
  (url varchar,  
   description text,  
   primary key (url)  
  )
```

```
create table Visit  
  (url_origin varchar,  
   visit_ref int,  
   date date,  
   primary key (url_origin, visit_id)  
  )
```

Une requête localisée et séquentielle

```
select * from Visit  
where url = 'sw.h.org'
```

Avec dénormalisation

Un type Visit

```
create type Visit (  
    number int,  
    date_visit date)
```

Imbriqué dans Origin

```
create table Origin  
    (url varchar,  
     description text,  
     visites list<Visit>,  
     primary key (url)  
    )
```

Suppose un nombre limité de visites.

Une requête localisée et unique pour trouver toutes les visites d'une origine.

```
select * from Origin  
where url ='swh.org'
```

La seconde visite

```
select * from Origin  
where url ='swh.org'  
and visit.number=2
```

En privilégiant l'accès Origin → Snapshot

```
create type Origin (  
  description int,  
  other_infos text)
```

```
create table Snapshot  
(url varchar,  
  snapshot_date,  
  snapshot_id uuid,  
  origin Origin,  
  visits map<date, Visit>,  
  branches list<string>,  
  primary key (url, snapshot_date)  
)
```

Suppose un nombre limité de visites et
de branches

Tous les snapshots d'une origine.

```
select * from Snapshot  
where url = 'sw.h.org'
```

Snapshots après une date et contenant
une branche master

```
select * from Snapshot  
where url = 'sw.h.org'  
and snapshot_date > '01-MARCH-2024'  
and branches contains key 'master'
```

Exemples pour Revision, Directory, Entry

Ici, vraie structure de graphe. À creuser.

```
create table Revision
(branch_id uuid,
 revision_id uuid,
 author varchar,
 message varchar,
 timestamp date,
 primary key (branch_id, revision_id)
)
```